Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU Assignment 7: More OpenMP

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: `to37gulo`
Student HPC-Account: `ptfs356v`

Gehan Younes
Student IdM: `qi39dyry`

Ahmed Mohamed
Student IdM: `gi10dafy`

June 24, 2026

1. OpenMP Fun (10 Credits)

The Problem: Race Condition on `tmp`

The variable `tmp` is declared outside the parallel region, so OpenMP classifies it as **shared** by default. Every thread executes the pair of statements:

```
double x[100], y[100], tmp;
int i;
double work(int);

#pragma omp parallel
{
  #pragma omp for
  for(i=0; i<100; i++) {
    tmp = work(i);        // RACE: all threads write the same variable
    x[i] = x[i] + tmp;  // RACE: thread may read another thread's tmp
  }
  #pragma omp for
  for(i=1; i<100; i++) {
    y[i] = x[i-1] * y[i];
  }
}
```

Listing 1: Original (buggy) code

Consider two threads A and B running concurrently. Thread A computes `tmp = work(0)` and is about to execute `x[0] = x[0] + tmp`. Before it does so, thread B overwrites `tmp = work(1)`. Thread A then reads the corrupted value of `tmp` and computes a wrong result for `x[0]`. This is a classic **write–read race condition** on the shared variable `tmp`. The result is non-deterministic and incorrect for any number of threads > 1.

The Fix: Make `tmp` Thread-Private

The cleanest fix is to declare `tmp` *inside* the loop body. A variable declared inside a parallel region is automatically **private** — each thread gets its own copy on the stack:

```
double x[100], y[100];
int i;
double work(int);

#pragma omp parallel
{
  #pragma omp for
  for(i=0; i<100; i++) {
    double tmp = work(i);  // private per thread: no race condition
    x[i] = x[i] + tmp;
  }
  #pragma omp for
  for(i=1; i<100; i++) {
    y[i] = x[i-1] * y[i];
  }
}
```

Listing 2: Corrected code: `tmp` declared locally

An equivalent fix is to add `private(tmp)` to the `#pragma omp parallel` directive, but declaring it locally is cleaner and less error-prone.

Why the Implicit Barrier Must Stay

The implicit barrier at the end of the first `#pragma omp for` is *essential for correctness* and must not be removed (e.g. with `nowait`). The second loop reads `x[i-1]`, which depends on

the fully updated `x[]` array from the first loop. Without the barrier, a thread could begin reading `x[i-1]` before another thread has finished writing it, producing a second, independent race condition.

2. π by the Monte Carlo Method (40 Credits)

Method

The quarter unit circle in the first quadrant has area $\pi/4$. By generating uniformly distributed random point pairs (x, y) in $[0, 1] \times [0, 1]$ and testing whether $x^2 + y^2 \leq 1$, the fraction of points inside the circle estimates $\pi/4$. With N points and c hits:

$$\pi \approx \frac{4c}{N}$$

(a) Parallel Implementation and Accuracy (20 Credits)

Key Design Decisions

Why `rand_r()` instead of `rand()`? `rand()` maintains a single global state shared across all threads. Concurrent calls without synchronisation cause a data race (undefined behaviour); adding a mutex serialises all calls and destroys parallelism. `rand_r()` takes a *per-thread seed pointer*, so each thread maintains a completely independent random sequence with no contention whatsoever. This is both correct and efficient.

Reduction for the counter. The `reduction(+:count)` clause creates a private copy of `count` for each thread; OpenMP sums all copies at the barrier at the end of the parallel region. This avoids false sharing and atomic contention on the counter.

Warmup parallel region. A bogus `#pragma omp parallel` block before the timed loop forces the OpenMP runtime to spin up all threads, so thread-creation overhead is not included in the benchmark time.

```
/* warmup: forces OpenMP to initialise thread pool before timing */
#pragma omp parallel
{ }

count = 0;
double wcstart = getTimeStamp();

#pragma omp parallel reduction(+:count)
{
    /* Each thread gets a unique seed -> independent random sequences */
    unsigned int seed = (unsigned int)(omp_get_thread_num() + 1)
                        * (unsigned int)time(NULL);
    #pragma omp for
    for (i = 0; i < N; i++) {
        double x = rand_r(&seed) * rcp;
        double y = rand_r(&seed) * rcp;
        if (x*x + y*y <= 1.0) count++;
    }
}
double pi = 4.0 * (double)count / (double)N;
```

Listing 3: Core parallelisation of `monte-carlo.c`

Result with 72 Threads and 4×10^9 Points

The program was compiled with the Intel `icx` compiler and run on a single Fritz node (Intel Xeon Platinum 8360Y, 72 physical cores).

```
PI      = 3.1415971370000002
Perf    = 4819.775 Mit/sec
Accuracy (relative error) = 1.4e-06
```

The achieved relative error is $\mathbf{1.4 \times 10^{-6}}$ (0.00014 %) using 72 threads and 4×10^9 sample points in double precision. The theoretical Monte Carlo convergence rate is $O(1/\sqrt{N})$; with $N = 4 \times 10^9$ this predicts an error of order $\sim 1/\sqrt{4 \times 10^9} \approx 1.6 \times 10^{-5}$, and the measured value is well within this bound.

(b) Performance Scaling on a Fritz ccNUMA Domain (20 Credits)

Experimental Setup

- Cluster: Fritz (FAU NHR), Intel Xeon Platinum 8360Y
- Cores tested: 1 to 18 (one ccNUMA domain)
- Clock speed: 2.4 GHz base (Fritz `powersave` governor)
- Points: $N = 4 \times 10^9$
- Compiler: `icx` with `-O3 -qopenmp`
- Job submitted via SLURM (`sbatch`)

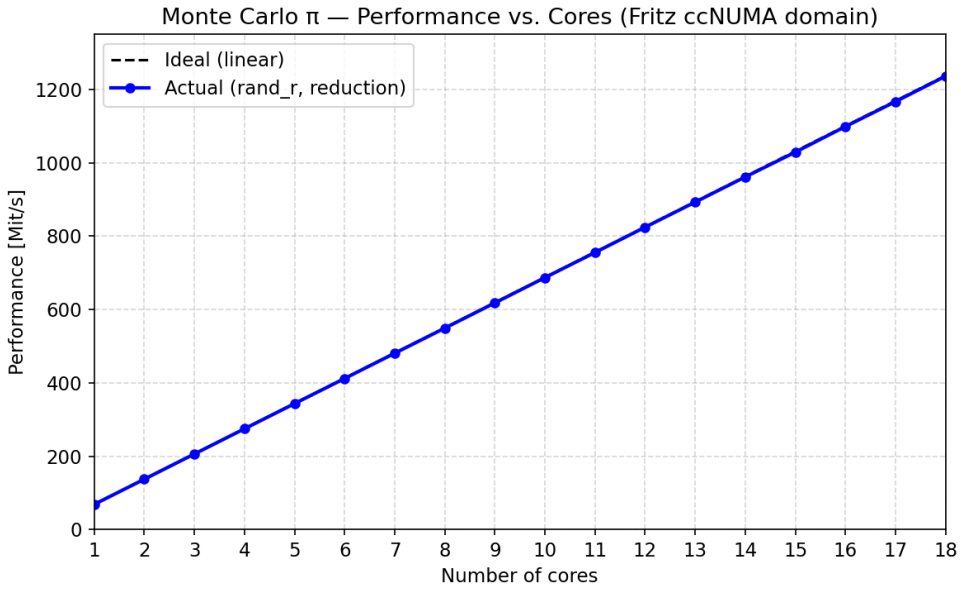


Figure 1: Monte Carlo π performance vs. number of cores on a Fritz ccNUMA domain (1–18 cores). Ideal linear scaling shown as a dashed reference line.

Results and Discussion

The code achieves **near-perfect linear scaling**: with 18 cores the measured performance is 1235.9 Mit/s versus the ideal $18 \times 68.7 = 1237.4$ Mit/s, giving a parallel efficiency of **99.9** %.

This near-ideal result follows directly from the design:

- `rand_r()` eliminates all shared state in the random number generation — no synchronisation is needed.
- `reduction(+:count)` avoids false sharing on the counter; each thread accumulates locally and the sum is performed once at the barrier.
- The work is perfectly load-balanced: `#pragma omp for` distributes the N iterations evenly.

- The warmup region eliminates OpenMP thread-startup latency from the measurement.

3. Parallel Ray Tracer (50 Credits)

Overview

The sequential ray tracer divides an image into rectangular tiles and calls `calc_tile()` for each. The original code uses a single shared `tile[]` buffer, which prevents direct parallelisation.

Parallelisation Strategy

The key change is to have `calc_tile()` write directly into the full `picture[]` buffer at index `picture[y*size + x]` instead of an intermediate tile buffer. Each pixel maps to a unique index, so no two threads ever write to the same memory location — no locks or atomics are required.

The tile loops are then parallelised with `collapse(2)` and `schedule(dynamic)`:

```
/* calc_tile now writes directly to picture[y*size + x].
   Each pixel is unique: no race conditions. */
void calc_tile(int size, int xstart, int ystart,
               int tilesize, char* picture)
{
  for (int y = ystart; y < ystart+tilesize; y++)
    for (int x = xstart; x < xstart+tilesize; x++) {
      /* ... ray computation ... */
      picture[y * size + x] = (char)c;
    }
}

/* Warmup to avoid measuring thread startup */
#pragma omp parallel
{
  #pragma omp single
  fprintf(stderr, "Running with %d threads\n", omp_get_num_threads());
}

#pragma omp parallel for collapse(2) schedule(dynamic)
for (int yc = 0; yc < ytiles; yc++)
  for (int xc = 0; xc < xtiles; xc++)
    calc_tile(size, xc*tilesize, yc*tilesize, tilesize, picture);
```

Listing 4: Core parallelisation of the ray tracer

`collapse(2)` flattens both tile loops into a single loop of $xtiles \times ytiles$ iterations, maximising the work pool that OpenMP can distribute. `schedule(dynamic)` handles load imbalance: tiles covering the sphere cluster (centre of the image) require significantly more ray-sphere intersection tests and recursive reflections than background tiles.

(a) Scaling with tilesize = 4500 (20 Credits)

Experimental Setup

- Image size: 18000×18000 pixels
- Tile size: 4500×4500 pixels $\Rightarrow 4 \times 4 = 16$ tiles total
- Cores tested: 1, 2, 4, 8, 16, 24, 32, 48, 64, 72
- Compiler: `icx -O3 -xHost -qopt-zmm-usage=high -fno-alias -qopenmp`
- Clock: 2.4 GHz base (Fritz `powersave` governor)

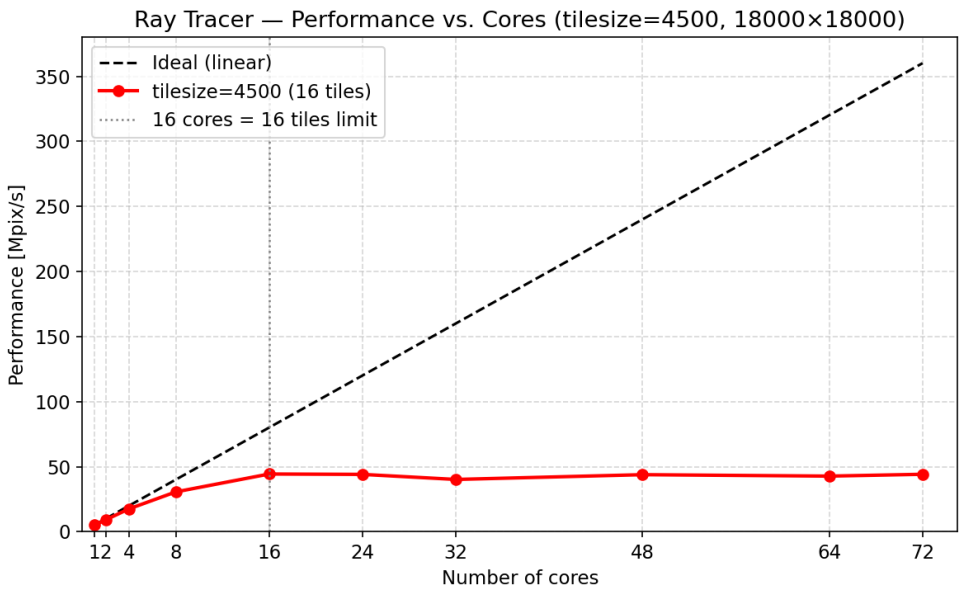


Figure 2: Ray tracer performance vs. number of cores with `tilesize=4500` (16 tiles total). Performance saturates at 16 cores because no more tiles are available.

Observations

Performance scales well from 1 to 16 cores (one tile per core), then **completely saturates**: with 16, 24, 32, 48, 64, and 72 cores the throughput stays at approximately 44 Mpix/s. With only 16 tiles and 72 cores, at most 16 cores are active at any time; the remaining 56 cores sit idle.

(b) Barrier Synchronisation Analysis (10 Credits)

The parallelised code contains **exactly one implicit OpenMP barrier**: at the end of the `#pragma omp parallel for` construct.

At a clock speed of 2.4 GHz:

$$T_{\text{barrier, GCC}} = \frac{50{,}000\,\text{cy}}{2.4 \times 10^9\,\text{Hz}} \approx 20.8\,\mu\text{s}$$

$$T_{\text{barrier, Intel}} = \frac{10{,}000\,\text{cy}}{2.4 \times 10^9\,\text{Hz}} \approx 4.2\,\mu\text{s}$$

The total runtime with 72 cores is approximately 2–3 seconds. A single barrier of 20.8 μs represents a fraction of $\sim 10^{-5}$ of the total runtime. **Barrier synchronisation is not a problem in either case**: the one-time barrier overhead is five orders of magnitude smaller than the computation time, making it completely negligible regardless of compiler. This can be verified experimentally: adding an explicit `#pragma omp barrier` inside the loop would increase the number of barriers to equal the number of tiles, and even then the total barrier overhead (e.g. $16 \times 20.8\,\mu\text{s} = 333\,\mu\text{s}$ for GCC) remains negligible compared to seconds of computation.

(c) Scaling to 72 Cores (20 Credits)

Root Cause of Poor Scaling in Part (a)

With `tilesize=4500`, the image is divided into only $\frac{18000}{4500} \times \frac{18000}{4500} = 4 \times 4 = 16$ tiles. With 72 threads and only 16 work units, at most 16 threads can be active simultaneously. Amdahl's law bounds the speedup to 16×, which is exactly what was observed.

Fix: Reduce Tile Size

By reducing `tilesize` to 250, the image is divided into $\frac{18000}{250} \times \frac{18000}{250} = 72 \times 72 = 5{,}184$ tiles — roughly 72 tiles per core at full occupancy. With `schedule(dynamic)`, the runtime assigns tiles to threads as they become free, ensuring good load balance even though centre tiles are heavier than background tiles.

Experimental Setup

- Image size: 18000×18000 pixels (unchanged)
- Tile size: 250×250 pixels $\Rightarrow$ 5,184 tiles
- `collapse(2)` + `schedule(dynamic)`
- All other settings identical to part (a)

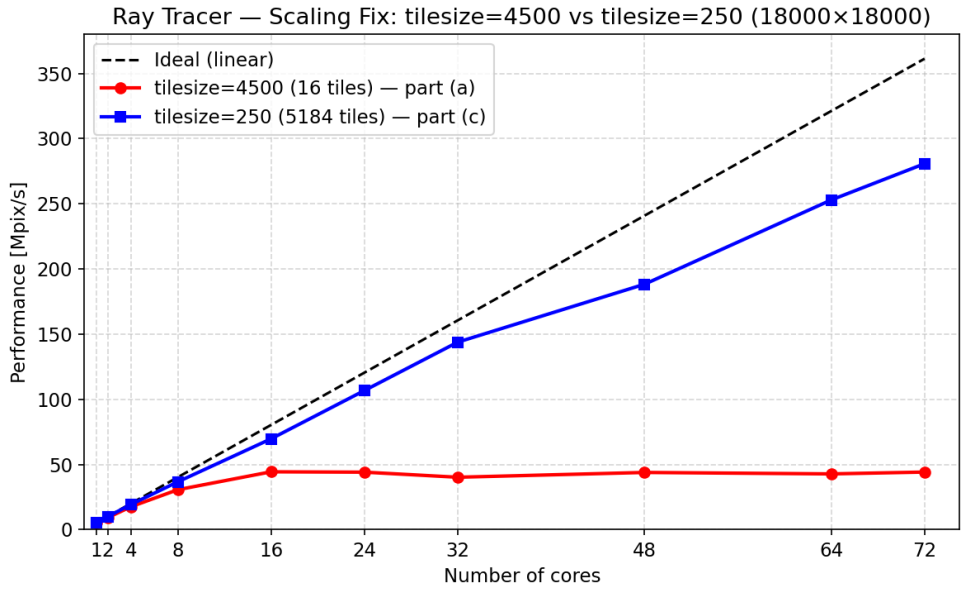


Figure 3: Scaling comparison: `tilesize=4500` (16 tiles, saturates at 16 cores) vs. `tilesize=250` (5184 tiles, continues scaling to 72 cores). Ideal linear scaling shown as dashed line.

Results

Cores	tilesize=4500 [Mpix/s]	tilesize=250 [Mpix/s]
1	5.001	5.018
2	9.144	9.957
4	17.565	19.287
8	30.578	36.615
16	44.323	69.647
24	44.049	106.706
32	40.156	143.841
48	43.825	188.247
64	42.717	252.917
72	44.130	280.735

Table 1: Ray tracer performance with `tilesize=4500` vs. `tilesize=250` on Fritz.

With `tilesize=250`, the 72-core speedup relative to sequential is:

$$S_{72} = \frac{280.735\,\text{Mpix/s}}{5.018\,\text{Mpix/s}} \approx \mathbf{55.9}\times \quad \Rightarrow \quad \text{parallel efficiency} = \frac{55.9}{72} \approx \mathbf{77.7}\,\%$$

The remaining gap from perfect 72× scaling is attributable to:

1. **Residual load imbalance**: tiles covering the sphere cluster in the image centre involve up to 10 recursive ray reflections, while background tiles terminate immediately. `schedule(dynamic)` mitigates but cannot eliminate this.
2. **NUMA effects**: Fritz nodes have 4 NUMA domains (4 memory controllers, 18 cores each). Threads on one domain that access memory allocated on another domain incur a latency and bandwidth penalty. The `picture[]` buffer is allocated on one NUMA node (first-touch policy), so threads on remote domains pay a higher memory access cost.